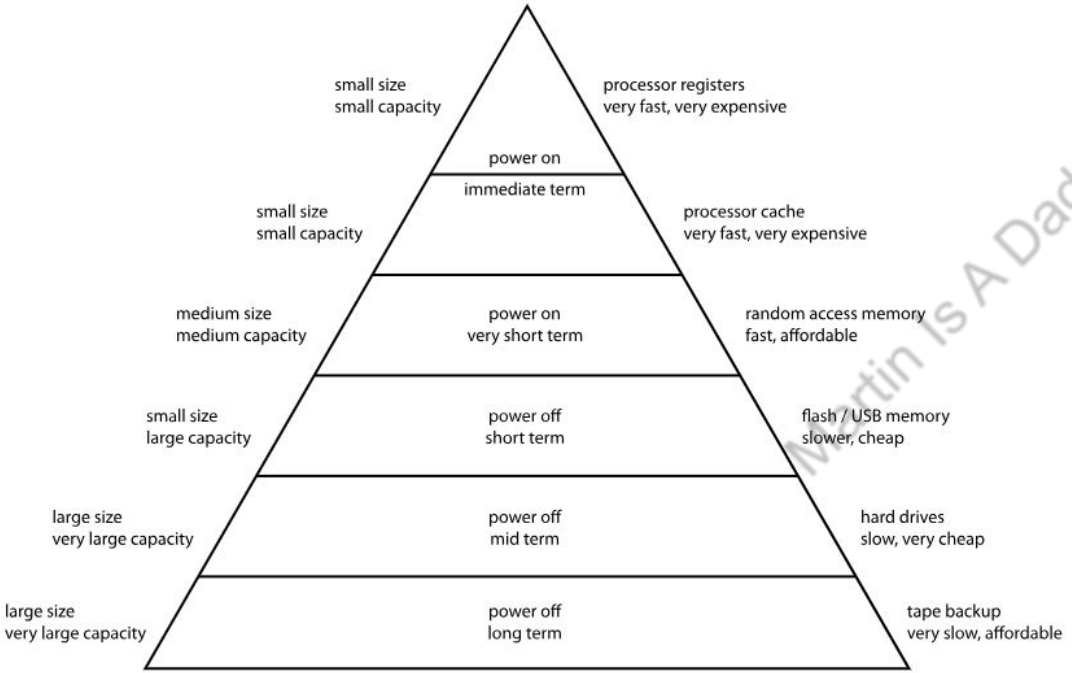
Before We Start

This talk is relatively math heavy, however

- It won't affect your ability to understand high level mechanism
- It won't stop you from using FlashAttention modules in development
- It'd be fun and helpful to follow even if you don't fully understand

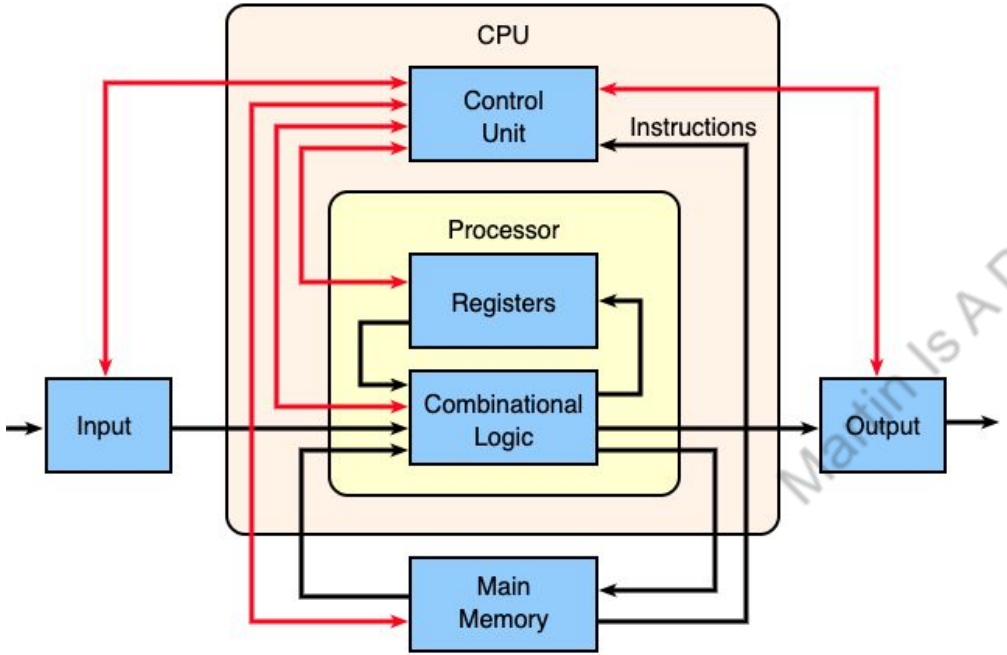
Just try to enjoy and have fun :D

How Does CPU and Memory Works?



- During DSA interviews, O() notations are used frequently for memory complexity. However, It could give you a false impression that memory is homogeneous, when it is actually hierarchical. It also focus on compute but neglecting I/O, which can be more critical for I/O bound performance.
- The computer memory hierarchy is a system where memory is organized into levels with varying **I/O** speeds, costs, and capacities.
- It typically consists of CPU registers, cache memory, main memory, secondary storage, and sometimes tape storage.
- This hierarchical structure allows for fast access to frequently used data while utilizing cheaper, slower storage for less frequently accessed information.

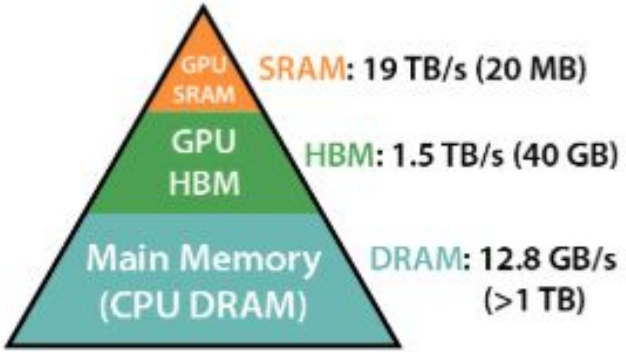
How Does CPU and Memory Works?



Block diagram of a basic uniprocessor-CPU computer. Black lines indicate data flow, whereas red lines indicate control flow; arrows indicate flow directions.

- To finish a computer program or instruction, CPU needs to
  - Fetch data from different level of memories (register -> cache -> main memory -> flash -> Disk etc)
  - Then compute using those data
- If a program/task is bottlenecked at fetching data from different level of memories, we call it I/O bound. To optimize I/O bound task, we can improve data locality, improve algorithm to reduce read/write or increase bandwidth with better hardware.
- If a program/task is bottlenecked at actual computation / data processing, we call it CPU bound. To optimize CPU bound task, we can optimize algorithm to reduce computation / processing need.

How Does GPU Memory Works?



Memory Hierarchy with Bandwidth & Memory Size

NVIDIA A100 40G GPU

- Similarly, GPU memory is also hierarchical. SRAM is fast (IO wise) and expensive, thus small capacity. HBM (high bandwidth memory) is cheaper and slower (still faster than DRAM) and larger capacity.
- When we say GPU is 40G or 80G, it usually refers to the HBM, since most of the operations that people cares rely on HBM.
- For I/O bound tasks, imagine if we can do more with SRAM and less with HBM, it would be a lot faster.
- *"On modern GPUs, compute speed has out-paced memory speed, and most operations in Transformers are bottlenecked by memory accesses."* **This is main motivation for FlashAttention.**

Transformer Recap

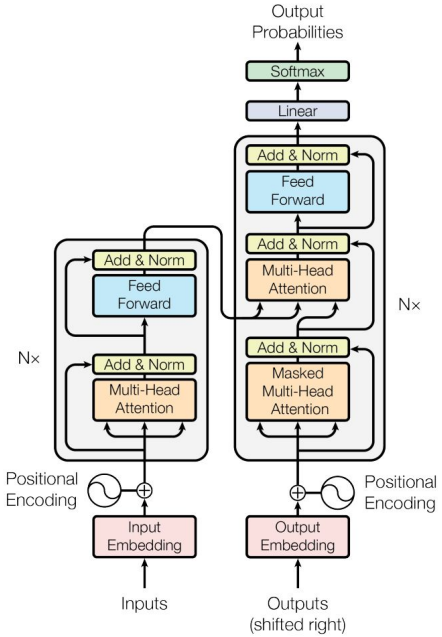
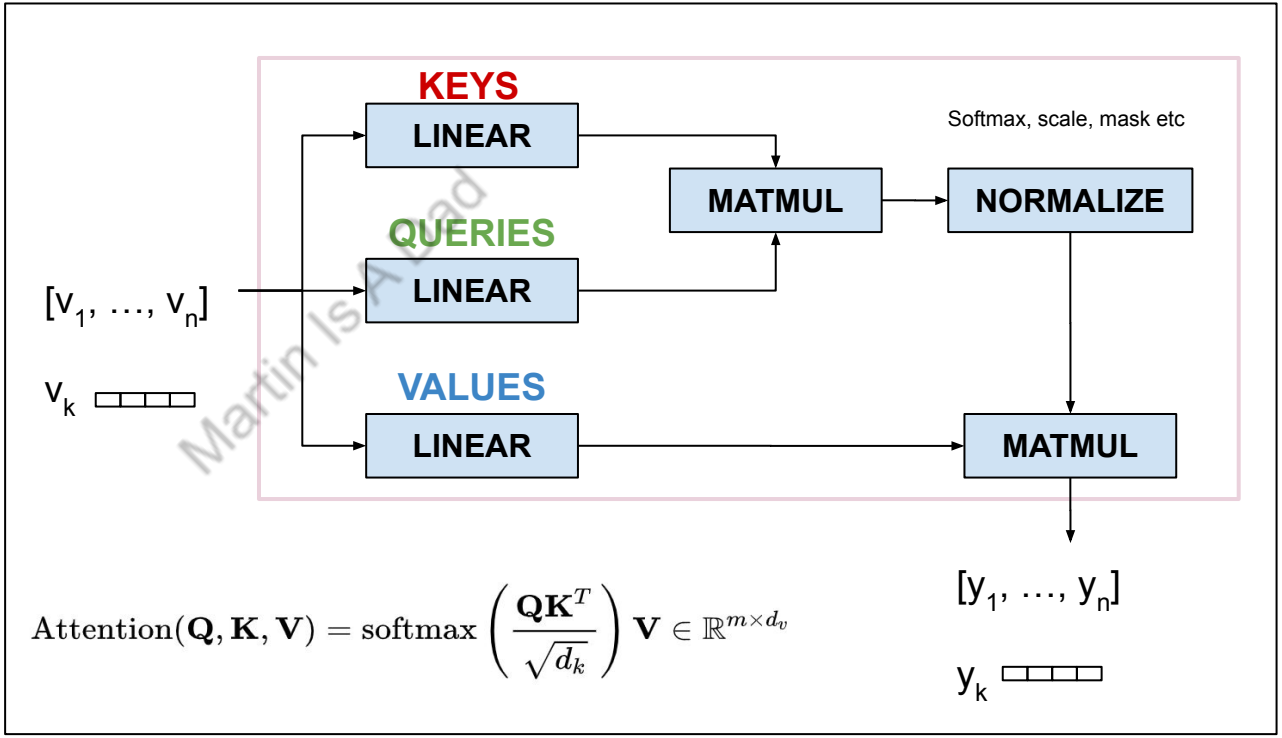


Figure 1: The Transformer - model architecture.



Standard Attention Implementation

- **Problem Statement**: Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

  - N is the sequence length. For large language models, the maximum sequence length can be very large (1 million for Gemini).
  - d is the head dimension for Multi-head Attention, usually a lot smaller (say 64 or 128)

- **Algorithm**

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Standard Attention Implementation

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

- **Memory I/O Analysis**
  - **Load Inputs**: **Q**, **K**, **V** matrices are read from HBM into SRAM. HBM Reads: 3×*Nd* elements.
  - **Compute Attention Scores** (**S=QK**^{T}): Matrix multiplication for an *N*×*N* matrix. This **S** matrix is often written back to HBM because it's too large to fit in SRAM for subsequent operations like softmax, especially for large *N*. HBM Writes: N^2 elements (for S).
  - **Compute Softmax** (**P**=softmax(**S**)): The **S** matrix is read from HBM. The resulting **P** matrix (also *N*×*N*) is computed and often written back to HBM for same reason. HBM Reads: N^2 elements (for **S**). HBM Writes: N^2 elements (for **P**).
  - **Compute Output** (**O=PV**): The **P** matrix and **V** matrix are read from HBM. The final output **O** (size *N*×*d*) is written to HBM. HBM Reads: N^2 +*Nd* elements (for **P** and **V**). HBM Writes: *Nd* elements (for **O**).
  - **Total IO**
    - $\boldsymbol{\Theta}(N^2 + Nd)$ HBM access ($\boldsymbol{\Theta}$() means tight bound, where ***O***() means worst case)

FlashAttention V1 High Level

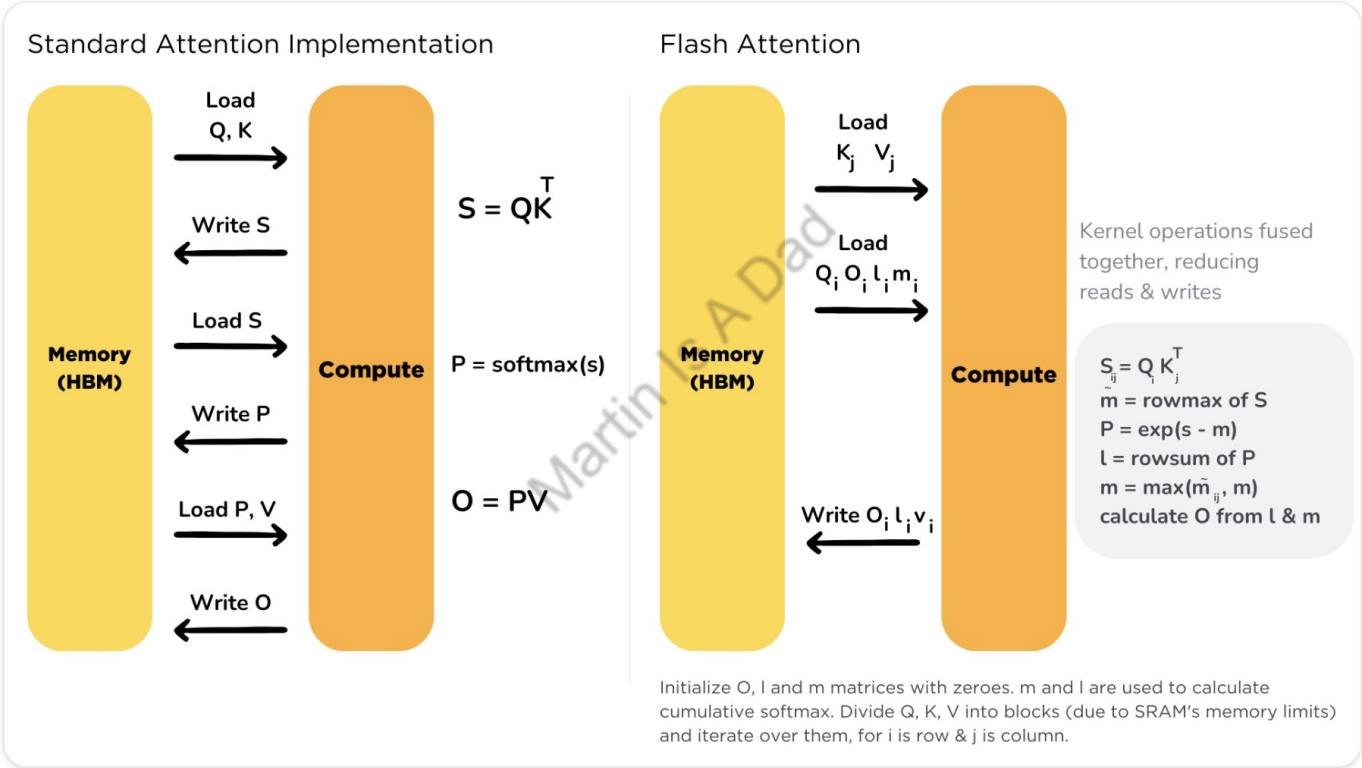
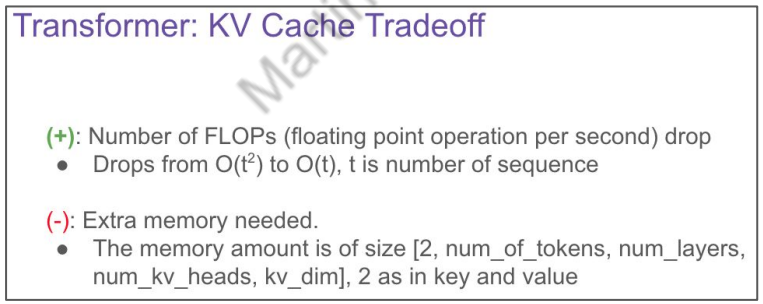


Image From [Huggingface Flash Attention Intro]

Extra Clarification

- **Training** vs **Inference**
  - FlashAttention's mechanism in principle can be used in inference (Flash Decoding etc). However in this talk's context we mainly focus on the training part.
  - For FlashAttention's usage in LLM inference, we might cover it in later episodes.

- **I/O Analysis** vs **FLOPs Analysis**
  - FlashAttention aims to improve memory I/O instead of FLOPs (floating point operation per second). So we are analyzing memory access (read/writes) instead of computation needed.
  - Remember we did FLOPs analysis when going through KV caching in LLM Inference, and it has nothing to do with FlashAttention



  - Btw, KV Caching is usually used in LLM inference, and not in LLM training
    - LLM inference is auto-regressive, thus KV cache is useful
    - LLM training's input sequence is processed together parallelly since we already know the input

Extra Background: Per Row Softmax

In the attention mechanism, specifically in scaled dot-product attention, softmax is calculated per row of the score matrix ($\mathbf{S}=\mathbf{Q}\mathbf{K}^T$) since each row corresponds to a single query vector and its relationship with all key vectors. The purpose is to determine how much attention that specific query should pay to every key.

- **Q** is the matrix of query vectors, dimension is *N*x*d* (length of sequence x length of attention heads)
- **K** is the matrix of key vectors, dimension is also *N*x*d*
- For each query vector in **Q**, we need to calculate the attention scores against all key vectors in **K**. The resulting matrix $\mathbf{S}=\mathbf{Q}\mathbf{K}^T$ has dimensions $N \times N$
  - The *i*-th row of **S**, denoted $\mathbf{S}_i$ contains the dot product scores of the *i*-th query vector $\mathbf{q}_i$ with all key vectors ($\mathbf{k}_1, \mathbf{k}_2, ..., \mathbf{k}_N$), where $\mathbf{S}_{ij}=\mathbf{q}_i \cdot \mathbf{k}_j$ represents the raw relevance / attention score of $\mathbf{q}_i$ and $\mathbf{k}_j$
  - Softmax is applied to all rows, so the sum of all elements in row $\mathbf{S}_i$ equals to 1, to represent probability

Softmax Definition: for a given input vector $\mathbf{S}=[\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_K]$, the softmax function calculates the probability $\mathbf{P}_i$ for each element $\mathbf{S}_i$ as follows:

$$P_i = \frac{e^{S_i}}{\sum_{j=1}^{K} e^{S_j}}$$

Extra Background: Safe Softmax

Given the Softmax definition $y_i = \frac{e^{x_i}}{\sum_{j=1}^{V} e^{x_j}}$, the implementation should be:

$d_0 \leftarrow 0$

1st pass

for $j \leftarrow 1, V$ do

$d_j \leftarrow d_{j-1} + e^{x_j}$

end for

2nd pass

for $i \leftarrow 1, V$ do

$y_i \leftarrow \frac{e^{x_i}}{d_V}$

end for

Naive Softmax Algorithm

The naive algorithm won't fly for most of the production environment, since e^{x_i} will overflow the hardware limit. We can use Safe Softmax $y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}}$

$m_0 \leftarrow -\infty$

1st pass

for $k \leftarrow 1, V$ do

$m_k \leftarrow \max(m_{k-1}, x_k)$

end for

$d_0 \leftarrow 0$

2nd pass

for $j \leftarrow 1, V$ do

$d_j \leftarrow d_{j-1} + e^{x_j - m_V}$

end for

3rd pass

for $i \leftarrow 1, V$ do

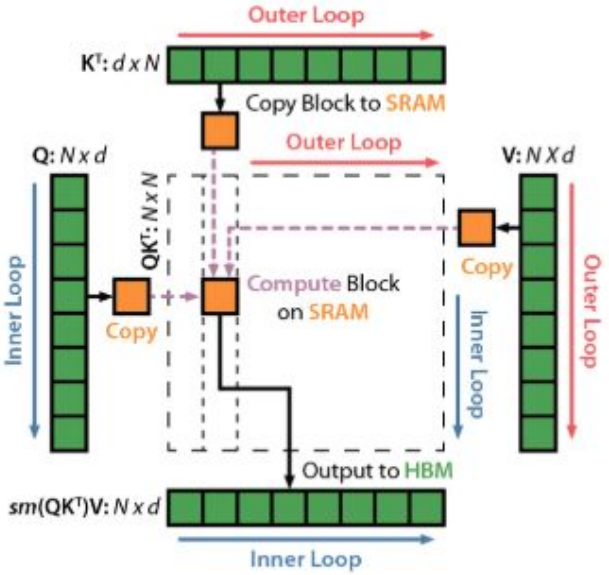
$y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$

end for

Safe Softmax Algorithm

FlashAttention V1: Tiling

Standard attention needs multiple HBM access, since **S** and **P** is too big for SRAM (with faster IO). FlashAttention aims to improve this by doing attention calculation with **tiling** (breaking big data into smaller tiles).



- **Divide and Conquer**: The input matrices (**Q**, **K**, **V**) are divided into smaller blocks or tiles.
- **Iterative Processing**: Instead of computing the full attention matrix at once, FlashAttention processes these tiles iteratively. It loads a block of Q and a block of K into the much faster SRAM.
- **Partial Computations**: It computes the attention scores and the weighted sum of values for these blocks within SRAM.
- **Online Softmax**: FlashAttention recomputes the softmax normalization "on-the-fly" as it processes more blocks. It correctly (no approximation) normalize the attention scores without needing the complete $\mathbf{QK^T}$ matrix.
- **Accumulation**: The results from processing each pair of Q and K blocks are accumulated to produce the final output, without ever forming the full intermediate attention matrix in HBM.

FlashAttention V1: Online Safe Softmax

Regular Safe Softmax won't work for FlashAttention since :

- It requires the complete row $\mathbf{S_i}$ to compute maximum value (say **m**), and subtract that maximum value when calculating softmax to avoid overflow.
- FlashAttention is doing tiling to fit into SRAM, so it no longer have access to the whole row

What should we do?

Algorithm 1 3-pass safe softmax

1: **Notations**
2: $\{m_i\} := \max_{j=1}^{i}\{s_j\}$, with initial value $m_0 = -\infty$.
3: $\{l_i\} : \sum_{j=1}^{i} e^{s_j - m_N}$, with initial value $l_0 = 0$, l_N is the denominator of safe softmax.
4: $\{p_i\}$: the final softmax value.
5:
6: **Body**
7: **for** $i \leftarrow 1, N$ **do**
8: $\quad m_i \leftarrow \max(m_{i-1}, s_i)$
9: **end for**
10:
11: **for** $i \leftarrow 1, N$ **do**
12: $\quad l_i \leftarrow l_{i-1} + e^{s_i - m_N}$
13: **end for**
14:
15: **for** $i \leftarrow 1, N$ **do**
16: $\quad p_i \leftarrow \frac{e^{s_i - m_N}}{l_N}$
17: **end for**

- 1st pass is friendly with tiling since for position *i*, it only uses information about *i* and *i-1*
- 2nd pass is not friendly with tiling due to dependency on m_N
- 3rd pass is not friendly with tiling due to dependency on m_N and l_N

What if we can find a substitute (denoted as $\tilde{l}_i$) for l_i so that:

- $\tilde{l}_i$ only depend on positions <= *i*
- $\tilde{l}_N = l_N$

Then we can avoid the 2nd pass, and make the algorithm closer to tiling compatible (still have m_N)

FlashAttention V1: Online Safe Softmax

- The substitute is found to be $\tilde{l}_i = \sum_{j=1}^{i} e^{s_j - m_i}$
- Given the recurrence relationship between $\tilde{l}_{i-1}$ and $\tilde{l}_i$

$$
\begin{aligned}
\tilde{l}_i &= \sum_{j=1}^{i} e^{s_j - m_i} \\
&= \left(\sum_{j=1}^{i-1} e^{s_j - m_i} \right) + e^{s_i - m_i} \\
&= \left(\sum_{j=1}^{i-1} e^{s_j - m_{i-1}} \right) e^{m_{i-1} - m_i} + e^{s_i - m_i} \\
&= \tilde{l}_{i-1} e^{m_{i-1} - m_i} + e^{s_i - m_i}
\end{aligned}
$$

- We can easily get the new 2-pass safe softmax algorithm, note that this is still **not** tiling friendly

Algorithm 2 2-pass safe softmax

for $i \leftarrow 1, N$ **do**
  $m_i \leftarrow \max(m_{i-1}, s_i)$
  $\tilde{l}_i \leftarrow \tilde{l}_{i-1} e^{m_{i-1} - m_i} + e^{s_i - m_i}$
end for

for $i \leftarrow 1, N$ **do**
  $p_i \leftarrow \frac{e^{s_i - m_N}}{\tilde{l}_N}$
end for

FlashAttention V1

- Recall $\mathbf{S} = \mathbf{QK}^\top \in \mathbb{R}^{N\times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N}, \quad \mathbf{O} = \mathbf{PV} \in \mathbb{R}^{N\times d}$
  - We have gone through **S** and **P**, now it's time to get the final result **O**
  - We will make it tiling friendly, and improve the algorithm further

- The naive algorithm based on the previous 2-pass Safe Softmax Algorithm is as follow:

Algorithm Multi-pass Self-Attention

Notations
$Q[k,:]$: the k-th row vector of Q matrix.
$K^T[:,i]$: the i-th column vector of K^T matrix.
$O[k,:]$: the k-th row vector of output O matrix.
$V[i,:]$: the i-th row of V matrix.
$\{o_i\} : \sum_{j=1}^{i} p_j V[j,:]$, a row vector storing partial aggregation result $P[k,:i] \times V[:i,:]$

Body
for $i \leftarrow 1, N$ **do**
  $s_i \leftarrow Q[k,:] \times K^T[:,i]$
  $m_i \leftarrow \max(m_{i-1}, s_i)$
  $\tilde{l}_i \leftarrow \tilde{l}_{i-1} e^{m_{i-1}-m_i} + e^{s_i - m_i}$
end for

for $i \leftarrow 1, N$ **do**
  $p_i \leftarrow \frac{e^{s_i - m_N}}{\tilde{l}_N}$
  $o_i \leftarrow o_{i-1} + p_i V[i,:]$
end for

$O[k,:] \leftarrow o_N$

- The 2-pass algorithm uses *k*-th row of **O** as example for simplicity. The computation of rows are **independent**.
- If we store the intermediate states (m_i p_i o_i and $\tilde{l}_i$), and yes we can store them in HBM given the small footprints & IO cost, then this algorithm is tiling friendly. We call it **Online Safe Softmax**
- Can we improve this algorithm further so it have 1 pass? **Yes**

FlashAttention V1

- Notice in the second pass $o_i = o_{i-1} + p_i V[i,:] = \sum_{j=1}^{i} p_j V[j,:] = \sum_{j=1}^{i} (\frac{exp(s_j - m_N)}{\tilde{l}_N} V[j,:])$

  it has dependency with m_N and $\tilde{l}_N$, which requires the first pass to finish. This is why we need 2 passes.

- What if we construct $\tilde{o}_i$ that meets the following criterias?
  - $\tilde{o}_i$ only depends on information from position <= *i*
  - $\tilde{o}_N$ = o_N

- With $\tilde{o}_i = \sum_{j=1}^{i} (\frac{exp(s_j - m_i)}{\tilde{l}_i} V[j,:])$, we can now merge 2 pass into 1, if we know the recurrence relationship between $\tilde{o}_i$ and $\tilde{o}_{i-1}$

$$
\begin{aligned}
\tilde{o}_i &= \sum_{j=1}^{i} \frac{e^{s_j - m_i}}{\tilde{l}_i} V[j,:] \\
&= \left(\sum_{j=1}^{i-1} \frac{e^{s_j - m_i}}{\tilde{l}_i} V[j,:] \right) + \frac{e^{s_i - m_i}}{\tilde{l}_i} V[i,:] \\
&= \left(\sum_{j=1}^{i-1} \frac{e^{s_j - m_{i-1}}}{\tilde{l}_{i-1}} \frac{\tilde{l}_{i-1}}{\tilde{l}_i} V[j,:] \right) + \frac{e^{s_i - m_i}}{\tilde{l}_i} V[i,:] \\
&= \left(\sum_{j=1}^{i-1} \frac{e^{s_j - m_{i-1}}}{\tilde{l}_{i-1}} V[j,:] \right) \frac{\tilde{l}_{i-1} e^{m_{i-1} - m_i}}{\tilde{l}_i} + \frac{e^{s_i - m_i}}{\tilde{l}_i} V[i,:] \\
&= \tilde{o}_{i-1} \frac{\tilde{l}_{i-1} e^{m_{i-1} - m_i}}{\tilde{l}_i} + \frac{e^{s_i - m_i}}{\tilde{l}_i} V[i,:]
\end{aligned}
$$

FlashAttention V1

Finally we have the 1 pass FlashAttention Algorithm that is tiling friendly:

Algorithm FlashAttention

for $i \leftarrow 1, N$ **do**

$$s_i \leftarrow Q[k,:] \times K^T[:,i]$$
$$m_i \leftarrow \max(m_{i-1}, s_i)$$
$$\tilde{l}_i \leftarrow \tilde{l}_{i-1} e^{m_{i-1}-m_i} + e^{s_i-m_i}$$
$$\tilde{o}_i \leftarrow \tilde{o}_{i-1} \frac{\tilde{l}_{i-1} e^{m_{i-1}-m_i}}{\tilde{l}_i} + \frac{e^{s_i-m_i}}{\tilde{l}_i} V[i,:]$$

end for

$$O[k,:] \leftarrow \tilde{o}_N$$

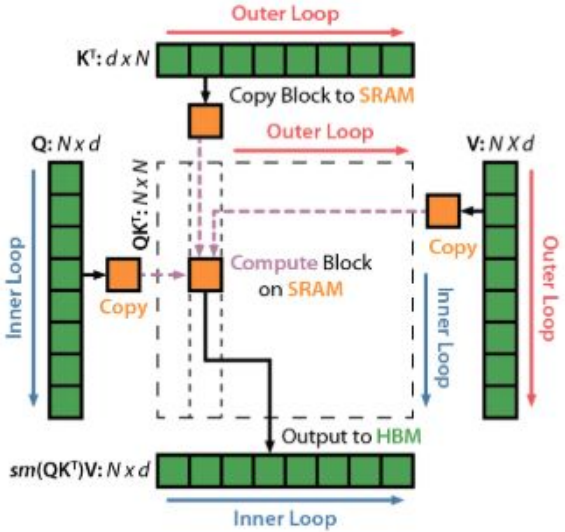
Now let's apply tiling to it!

FlashAttention V1

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

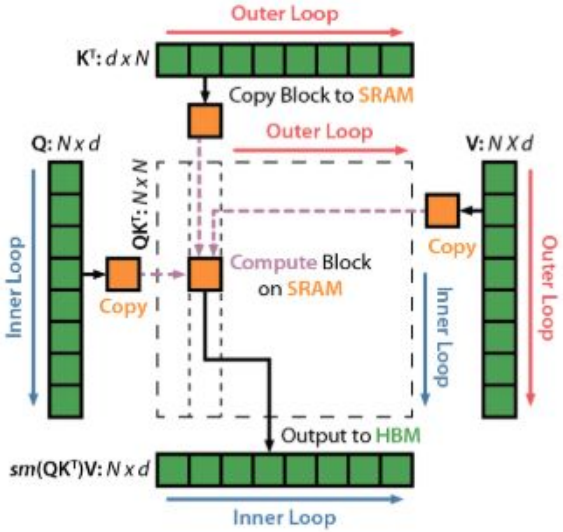


- diag() is constructing a diagonal matrix given vector
- upper bracket **⌈x⌉** is ceiling function, denoted by, which rounds a real number up to the nearest integer. For example, ⌈2.7⌉ = 3
- The outer loops is on **K** and **V**, and inner loop is on **Q**. This was later swapped for parallelism in V2.
- (***m***, ***ℓ***) are written to HBM for backward pass's recomputation

FlashAttention V1

- B_c and B_r block size are picked intentionally to fit in all 4 blocks of **Qi**, **Kj**, **Vj** and **S** into SRAM (of size ***M***)
  - Dimension of **S**: $B_r \times B_c$ <= min(d, $M/4d$) × $M/4d$ <= $d \times M/4d$ = M/4
  - You can design other block size, as long as all 4 matrices can fit into SRAM

- **Memory I/O Analysis**
  - In the outer loop, each element of **K** and **V** is loaded from HBM once, both are $N \times d$ dimensions, resulting in $2 \times Nd$ HBM reads. $\Theta(Nd)$
  - Given K and V are broken into $T_c = \lceil N/B_c \rceil$ blocks in the outer loop,we make T_c passes in the inner loop over **Q** and **O**, each pass loading all of **Q** and all of **O** to HBM. This results in $2 \times T_c \times Nd$ HBM reads. $\Theta(NdT_c)$
  - Total I/O complexity is $\Theta(NdT_c + Nd) = \Theta(NdT_c)$. We can further simplify this.



FlashAttention V1

- **Memory I/O Analysis**
  - Total I/O complexity is $\boldsymbol{\Theta}(NdT_c + Nd) = \boldsymbol{\Theta}(NdT_c)$. We can further simplify this.
  - $\mathbf{K}_j$, $\mathbf{V}_j$ needs to fit into SRAM (size of M). Dimension of $\mathbf{K}_j$, $\mathbf{V}_j$ is $\boldsymbol{B_c} \times d$, thus:

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right)$$

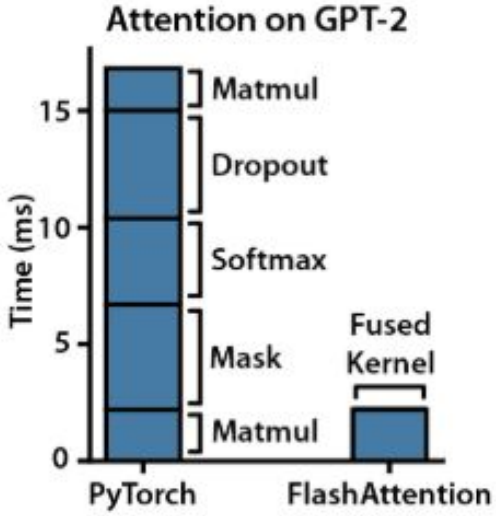
  - $\mathbf{Q}_i$ needs to fit into SRAM (size of M). Dimension of $\mathbf{Q}_i$ is $\boldsymbol{B_r} \times d$, thus:

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right)$$

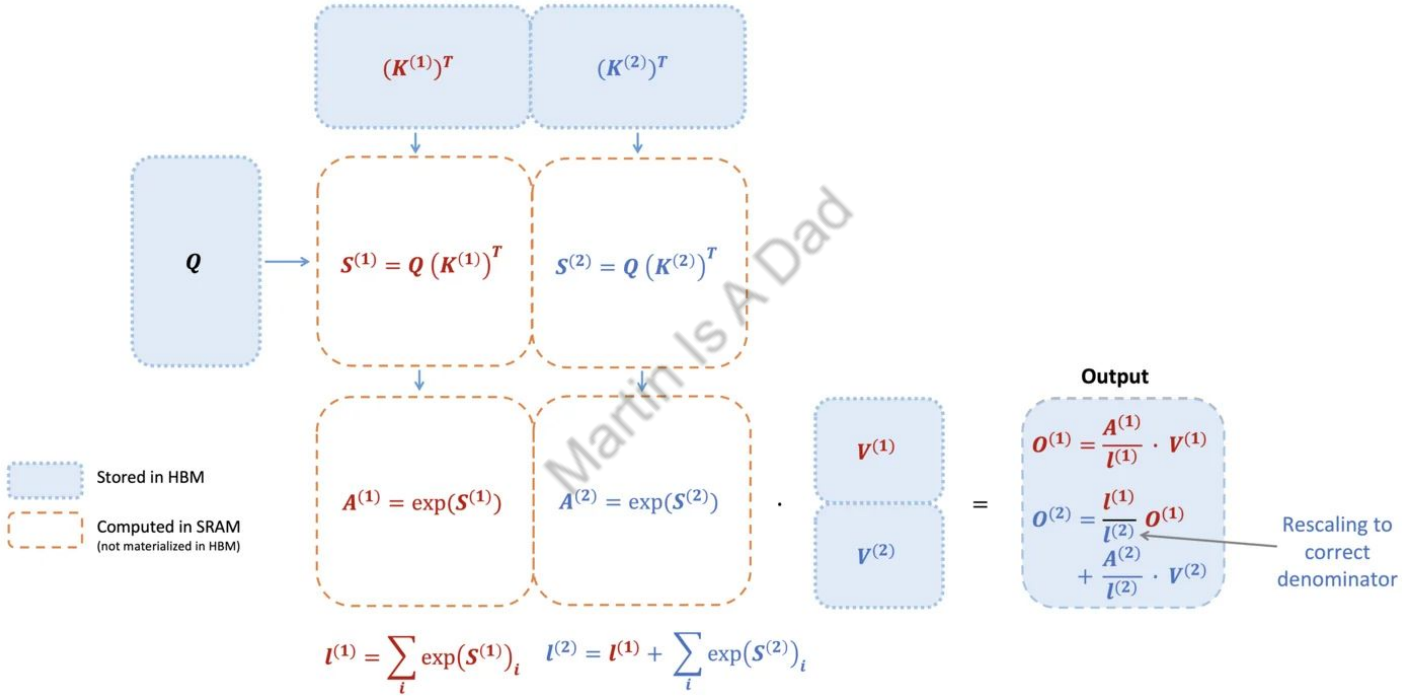
  - $\mathbf{S}_{ij}$ needs to fit into SRAM (size of M). Dimension of $\mathbf{S}_{ij}$ is $\boldsymbol{B_r} \times \boldsymbol{B_c}$, thus:

$$B_r B_c = O(M) \qquad B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right)$$

  - Then we have $T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right)$ and the final I/O complexity for FlashAttention V1 is $\Theta(NdT_c) = \Theta\left(\frac{N^2d^2}{M}\right)$
  - Comparing with the I/O complexity of standard attention $\boldsymbol{\Theta}(N^2)$, since typical value of d is small (64 or 128) and typical value of M (in KBs or MBs) is much larger, **FlashAttention requires many times fewer HBM accesses, leading to faster execution and lower memory footprint**



FlashAttention V1: Intuition



Intuition by paper's author Tri Dao in https://tridao.me/blog/2024/flash3/

Attention Backward Process

- You might notice that we have only gone through the forward process for Attention. With the previous knowledge that I have shared, you should already:
  - Know FlashAttention's motivation/essence, that is reduce memory I/O complexity
  - Know FlashAttention's mechanism including tiling, online softmax algorithm, etc
  - Comfortable using FlashAttention modules in development

- So this part about backward process is optional, stay if you feel like it.

- Let's recap what **backward propagation** is doing. The goal for training a neural network is to minimize a loss function (say ϕ). According to the chain rule, we use backpropagation to update weights for all intermediate neurons in the network.
  - Assume forward pass is done, and loss function is calculated for the output.
  - Backpropagation starts at the end of the network, the gradient of the loss function with respect to the network's final output is calculated first, say $\mathbf{dO}_{\mathbf{final}}$
  - For any given intermediate layer, it takes the succeeding layer's **dO** as input, and calculate current layer's updated parameters

- **Problem Statement:** Given loss function ϕ, and let the output gradient be $\mathbf{dO} \in \mathrm{R}^{n \times d}$ (where **dO** denotes $\partial\phi/\partial\mathrm{O}$). We want to compute the input gradients **dQ**, **dK**, **dV** $\in \mathrm{R}^{n \times d}$ (where **dQ**, **dK**, **dV** denote $\partial\phi/\partial\mathrm{Q}$, $\partial\phi/\partial\mathrm{K}$, $\partial\phi/\partial\mathrm{V}$ respectively)

Chain Rule

- We know $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ $\mathbf{O} \in \mathbb{R}^{N \times d}$ $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$
- We know $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}$, $\mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}$, $\mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}$

- $\partial\phi/\partial V = \partial\phi/\partial O \times \partial O/\partial V = \partial\phi/\partial O \times P$. Thus we hav $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$
- $\partial\phi/\partial P = \partial\phi/\partial O \times \partial O/\partial P = \partial\phi/\partial O \times V$. Thus we hav $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$
- $\partial\phi/\partial Q = \partial\phi/\partial S \times \partial S/\partial Q = \partial\phi/\partial S \times K$. Thus we hav $\mathbf{dQ} = \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N \times d}$
- $\partial\phi/\partial K = \partial\phi/\partial S \times \partial S/\partial K = \partial\phi/\partial S \times Q$. Thus we hav $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q} \in \mathbb{R}^{N \times d}$

Derivative of the Softmax Function

Softmax function takes a vector input and produces a vector output, its derivative is a **Jacobian matrix**. The Jacobian matrix contains all possible partial derivatives of each output element with respect to each input element.

Let $\mathbf{y} = \text{softmax}(\mathbf{z})$. We need to compute $\frac{\partial y_i}{\partial z_j}$ for all $i, j \in \{1, \ldots, K\}$.

There are two cases to consider:

Case 1: $i = j$ (Derivative of y_i with respect to its own input z_i)

$$y_i = \frac{e^{z_i}}{\sum_{k=1}^{K} e^{z_k}}$$

Let $S = \sum_{k=1}^{K} e^{z_k}$. Then $y_i = \frac{e^{z_i}}{S}$.

We can use the quotient rule for differentiation: $\frac{d}{dx}\left(\frac{u}{v}\right) = \frac{u'v - uv'}{v^2}$

Here, $u = e^{z_i}$ and $v = S$.

So, $\frac{\partial u}{\partial z_i} = e^{z_i}$ and $\frac{\partial v}{\partial z_i} = e^{z_i}$.

$$\frac{\partial y_i}{\partial z_i} = \frac{e^{z_i} \cdot S - e^{z_i} \cdot e^{z_i}}{S^2}$$
$$= \frac{e^{z_i}}{S} \cdot \frac{S - e^{z_i}}{S}$$
$$= y_i \cdot \left(1 - \frac{e^{z_i}}{S}\right)$$
$$= y_i(1 - y_i)$$

Case 2: $i \neq j$ (Derivative of y_i with respect to a different input z_j)

$$y_i = \frac{e^{z_i}}{\sum_{k=1}^{K} e^{z_k}}$$

Again, let $S = \sum_{k=1}^{K} e^{z_k}$.

Now, when differentiating with respect to z_j (where $j \neq i$):

- $\frac{\partial u}{\partial z_j} = \frac{\partial (e^{z_i})}{\partial z_j} = 0$ (since e^{z_i} does not depend on z_j)
- $\frac{\partial v}{\partial z_j} = \frac{\partial S}{\partial z_j} = e^{z_j}$ (since z_j is a term in the sum S)

Applying the quotient rule:

$$\frac{\partial y_i}{\partial z_j} = \frac{0 \cdot S - e^{z_i} \cdot e^{z_j}}{S^2}$$
$$= -\frac{e^{z_i} e^{z_j}}{S^2}$$
$$= -\left(\frac{e^{z_i}}{S}\right)\left(\frac{e^{z_j}}{S}\right)$$
$$= -y_i y_j$$

Derivative of the Softmax Function

Combining both cases, the partial derivative $\frac{\partial y_i}{\partial z_j}$ is:

$$\frac{\partial y_i}{\partial z_j} = \begin{cases} y_i(1 - y_i) & \text{if } i = j \\ -y_i y_j & \text{if } i \neq j \end{cases}$$

The full Jacobian matrix J of the Softmax function is then:

$$J = \begin{pmatrix} y_1(1 - y_1) & -y_1 y_2 & \dots & -y_1 y_K \\ -y_2 y_1 & y_2(1 - y_2) & \dots & -y_2 y_K \\ \vdots & \vdots & \ddots & \vdots \\ -y_K y_1 & -y_K y_2 & \dots & y_K(1 - y_K) \end{pmatrix}$$

This can also be written more compactly using vector notation:

$$J = \text{diag}(\mathbf{y}) - \mathbf{y}\mathbf{y}^T$$

Derivative of the Softmax Function

- We know that $\mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}$
- Given the Jacobian matrix for y = softmax(x) is diag(**y**) - **y**$\mathbf{y}^T$, we have

$$dS_{i:} = (\text{diag}(P_{i:}) - P_{i:}P_{i:}^T)dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:})P_{i:}$$

$$dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$$

Standard Attention Backward Implementation

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

1: Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
2: Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
3: Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
4: Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dSK}$, write $\mathbf{dQ}$ to HBM.
5: Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
6: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

The backward pass typically requires the matrices **S**, **P** $\in \mathrm{R}^{N \times N}$ to compute the gradients with respect to **Q**, **K**, **V**. Just like the forward pass, due to the size of the matrix, we need to do lots of read and write to HBM.

- **One of FlashAttention's goals is to not store $O(N^2)$ intermediate values for the backward pass**. By storing the output **O** and the softmax normalization statistics (m, ℓ), we can recompute the attention matrix **S** and **P** in the backward pass from blocks of **Q**, **K**, **V, O and** (m, ℓ) in SRAM.

- This results in more FLOPs due to recomputation, however, it still speeds up the backward pass due to reduced HBM accesses

FlashAttention V1 Backward Implementation

Algorithm 4 FlashAttention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

Set the pseudo-random number generator state to $\mathcal{R}$.
Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}, \mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop}.
    On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
    On chip, compute $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

Recompute with m and ℓ, formula same with forward pass

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i$$

$$dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$$

Define

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j = do_i^T o_i$$

Then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}$$

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i)$$

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) k_j$$

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) q_i$$

FlashAttention V1 Backward I/O Complexity

Algorithm 4 FlashAttention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

1: Set the pseudo-random number generator state to $\mathcal{R}$.
2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
5: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
6: **for** $1 \le j \le T_c$ **do**
7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
8: Initialize $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}, \mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ on SRAM.
9: **for** $1 \le i \le T_r$ **do**
10: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
11: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
12: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
13: On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
14: On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop}.
15: On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
16: On chip, compute $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
17: On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
18: On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
19: On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
20: On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
21: Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
22: On chip, compute $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
23: **end for**
24: Write $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ to HBM.
25: **end for**
26: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

Recompute with ***m*** and ***ℓ***, formula same with forward pass

Memory I/O Analysis

- Very similar to forward process
- In the outer loop, each element of **K** and **V** is loaded from HBM once, both are $N \times d$ dimensions, resulting in $2 \times Nd$ HBM reads. $\boldsymbol{\Theta}(Nd)$
- Given K and V are broken into $T_c = \lceil N/B_c \rceil$ blocks in the outer loop,we make T_c passes in the inner loop over **Q**, **O** and **dO**, each pass loading all of **Q**, **O** and **dO** to HBM. This results in $3 \times T_c \times Nd$ HBM reads. $\boldsymbol{\Theta}(NdT_c)$
- Total I/O complexity is $\boldsymbol{\Theta}(NdT_c + Nd) = \boldsymbol{\Theta}(NdT_c)$.

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2 d^2}{M}\right)$$